

Πόρισμα Πειραμάτων Neural LSH στο MNIST

Υπερπαράμετροι Build 1^ο Μέρος

Πειραματική ρύθμιση

Για το MNIST χρησιμοποιήθηκε η υλοποίηση Neural LSH της 2ης εργασίας, με τα βήματα: (α) κατασκευή k-NN γράφου με εξωτερικό ANN αλγόριθμο, (β) ισοκατανεμημένη διαμέριση με KaHIP σε m μέρη, (γ) εκπαίδευση MLP ταξινομητή που προβλέπει το partition label, και (δ) multi-probe αναζήτηση πάνω στο inverted file.

Η αξιολόγηση έγινε όπως και στην 1η εργασία, με τις ίδιες μετρικές: Average AF (πρέπει να είναι κοντά στο 1), Recall@N, QPS και χρόνους tApproximateAverage / tTrueAverage. Στα πειράματα εδώ κρατήσαμε σταθερά:

- αριθμός partitions: $m = 100$
- KaHIP: imbalance $\varepsilon = 0.03$, mode = STRONG (2), seed = 1
- MLP: epochs = 10, lr = 0.001, seed = 1
- Search: $N = 1$, $T = 5$, $R = 2000$ (range = 2000, όπως στα MNIST πειράματα της 1ης εργασίας).

Οι υπερπαράμετροι που σαρώθηκαν για το build του Neural LSH στο MNIST ήταν:

- k (για τον k-NN γράφο): {1, 10, 50}
- Αριθμός στρωμάτων MLP (layers): {2, 3, 4}
- Κόμβοι ανά κρυφό στρώμα (nodes): {64, 128, 256}
- Μέγεθος batch: {64, 128, 256}

Σε όλα τα runs ισχύει $t_{\text{Approx}} < t_{\text{True}}$, άρα η προσεγγιστική μέθοδος είναι ουσιαστικά πιο γρήγορη από την εξαντλητική αναζήτηση.

Γενική εικόνα αποτελεσμάτων

Σε επίπεδο συνολικής στατιστικής:

- $AF \in [1.00, \sim 1.30]$, με μέσο όρο ≈ 1.08 .
- Recall@N από ~ 0.05 (πολύ κακές ρυθμίσεις) έως ~ 0.96 (βέλτιστες ρυθμίσεις).
- $QPS \approx 217\text{--}293$ (μέσο ≈ 255 QPS).
- $t_{\text{Approx}} \approx 3.4\text{--}4.6$ ms ανά query, ενώ $t_{\text{True}} \approx 38\text{--}39$ ms.
- Δηλαδή το Neural LSH δίνει επιτάχυνση περίπου $\times 9\text{--}11$ σε σχέση με την ακριβή αναζήτηση, με πολύ καλό AF (κοντά στο 1) στις σωστές ρυθμίσεις.

Επίδραση των υπερπαραμέτρων

k του k-NN γράφου (knn)

- Για $k = 1$ ο γράφος είναι υπερβολικά αραιός -> KaHIP βρίσκει partitions που δεν αντικατοπτρίζουν καλά τη γεωμετρία του χώρου.
- Μέγιστο Recall@N ≈ 0.35 , AF $\sim 1.09-1.30$ -> πρακτικά απορρίπτονται.
- Για $k = 10$ και $k = 50$ όλα τα runs έχουν $AF \leq 1.05$, με μέσο Recall@N ≈ 0.94 και πολύ παρόμοιους χρόνους.
- Το $k = 50$ δίνει οριακά το μέγιστο Recall (0.9603), αλλά το $k = 10$ είναι ελαφρύτερο στην κατασκευή του γράφου (λιγότερο κόστος build) με σχεδόν ίδια ποιότητα.

Αριθμός στρωμάτων και κόμβων του MLP

- Αυξάνοντας layers από 2 -> 3 -> 4 και nodes από 64 -> 128 -> 256, το Recall βελτιώνεται και το AF πλησιάζει περισσότερο το 1, με μικρή μόνο επιβάρυνση στο QPS.
- Τα καλύτερα runs συγκεντρώνονται στα layers $\in \{3, 4\}$ και nodes = 256 (μεγάλο, «ισχυρό» MLP).

Batch size

- Batch size = 64, 128, 256 δίνουν παρόμοιο QPS ($\sim 254 - 256$ QPS κατά μέσο όρο), με μικρές διαφορές στους χρόνους.
- Στα κορυφαία runs εμφανίζεται σχεδόν πάντα batch_size = 256.

Καλύτερες ρυθμίσεις Neural LSH στο MNIST

Ως κριτήριο επιλογής «βέλτιστων» ρυθμίσεων χρησιμοποιήσαμε:

μέγιστο Recall@N -> ελάχιστο $|AF - 1|$ -> μέγιστο QPS,
με επιπλέον φίλτρο $AF \leq 1.05$ (καλή προσέγγιση).

Οι κορυφαίες ρυθμίσεις είναι:

1. Accuracy-first (μέγιστο Recall)

- knn = 50, m = 100, imbalance = 0.03, kahip_mode = 2
- MLP: layers = 4, nodes = 256, epochs = 10, batch_size = 256, lr = 0.001, seed = 1

Metrics:

- Recall@N = 0.9603
- AF = 1.0027
- QPS ≈ 243.52
- tApprox ≈ 4.11 ms, tTrue ≈ 39.28 ms

Δίνει την καλύτερη Recall σχεδόν χωρίς απώλεια στην ποιότητα ($AF \approx 1$). Το κόστος είναι ελαφρώς μικρότερο QPS και πιο «βαρύ» training ($k = 50$, μεγάλο MLP).

2. Balanced (προτεινόμενη ρύθμιση για τελικές συγκρίσεις)

- knn = 10, m = 100, imbalance = 0.03, kahip_mode = 2
- MLP: layers = 3, nodes = 256, epochs = 10, batch_size = 256, lr = 0.001, seed = 1

Metrics:

- Recall@N = 0.9588
- AF = 1.0029
- QPS $\approx$ 254.30
- tApprox $\approx$ 3.93 ms, tTrue $\approx$ 39.05 ms

Το Recall είναι πρακτικά ίδιο με την accuracy-first ρύθμιση (0.9588 vs 0.9603), AF $\approx$ 1, ενώ:

- το QPS είναι υψηλότερο,
- ο k-NN γράφος είναι πιο φθηνός στην κατασκευή (k = 10 αντί για 50),
- το MLP είναι ελαφρώς μικρότερο (3 layers αντί για 4).

Αυτή η ρύθμιση είναι η πιο ισορροπημένη και προτείνεται ως default configuration για τις συγκρίσεις Neural LSH vs LSH / Hypercube / IVFFlat / IVFPQ.

3. High-throughput (μέγιστο QPS με υψηλή Recall)

- knn = 10, m = 100, imbalance = 0.03, kahip_mode = 2
- MLP: layers = 4, nodes = 64, epochs = 10, batch_size = 128, lr = 0.001, seed = 1

Metrics:

- Recall@N = 0.9519
- AF = 1.0037
- QPS $\approx$ 289.02 (το μέγιστο στο grid)
- tApprox $\approx$ 3.46 ms, tTrue $\approx$ 37.92 ms

Εδώ θυσιάζουμε λίγο Recall (από ~ 0.96 σε ~ 0.95) αλλά κερδίζουμε το μέγιστο throughput. Είναι χρήσιμη ρύθμιση αν θέλουμε να δείξουμε το «ταχύτερο» Neural LSH στο MNIST.

Συνοπτικά συμπεράσματα

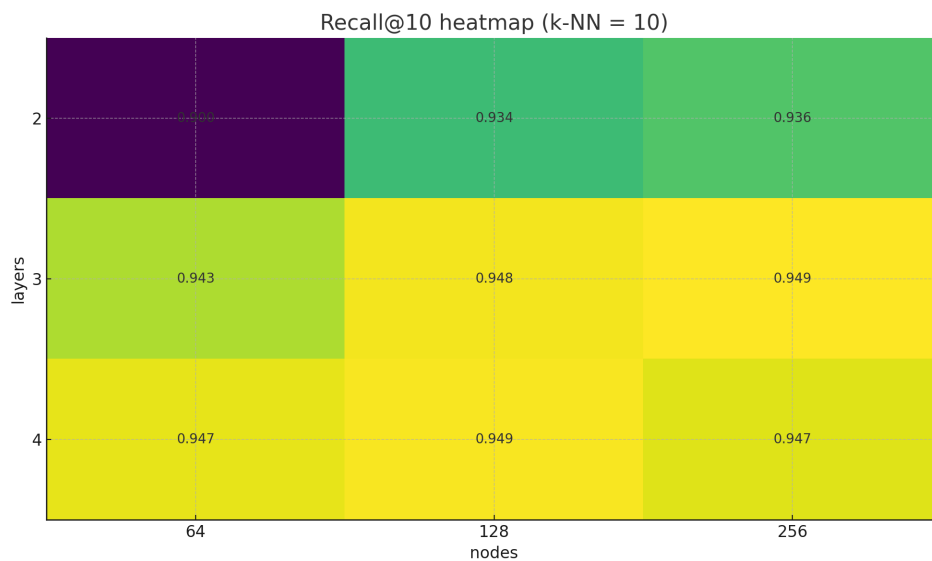
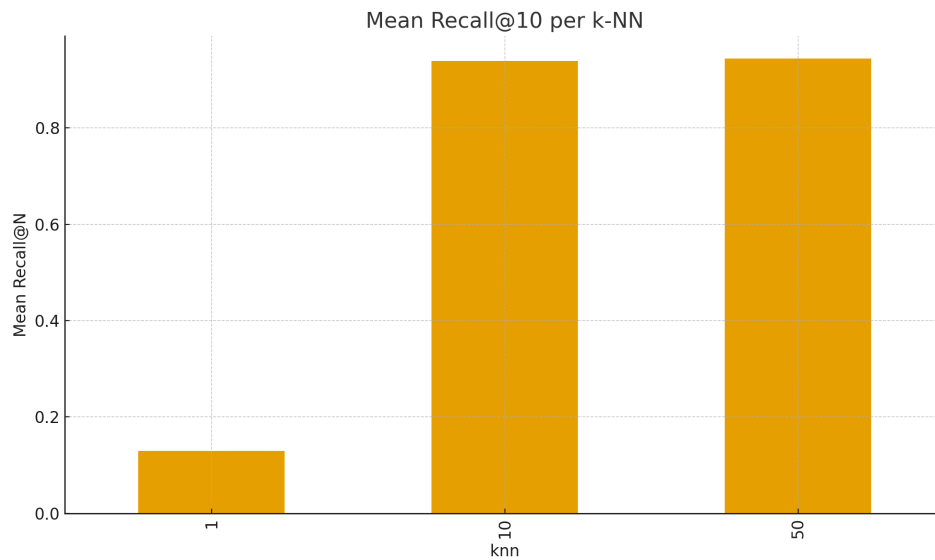
Το Neural LSH στο MNIST, με καλά επιλεγμένες υπερπαραμέτρους, πετυχαίνει Recall@N $\approx$ 0.96 με AF $\approx$ 1.00 και QPS $\approx$ 240 – 255, δηλαδή πολύ κοντά σε ακριβή NN με 8 – 10× μικρότερο χρόνο αναζήτησης.

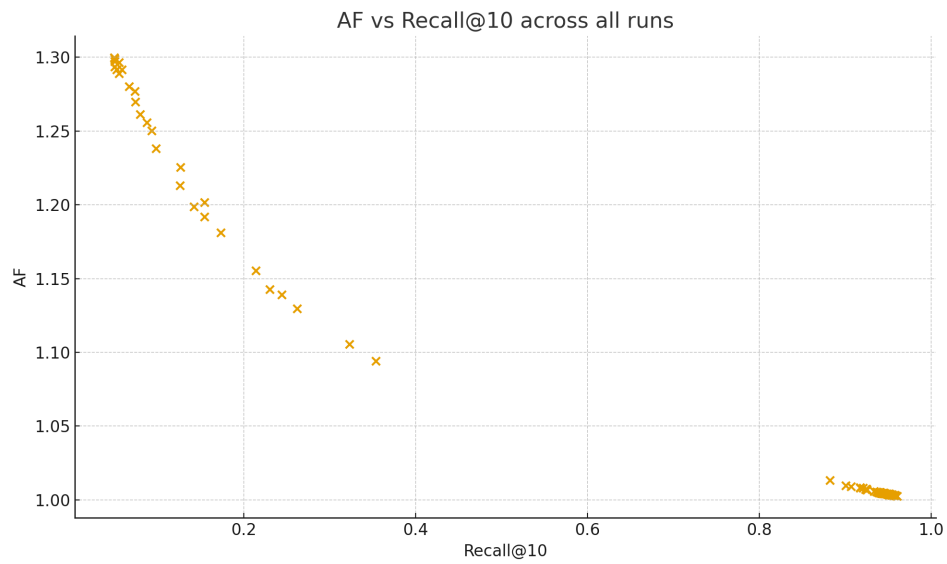
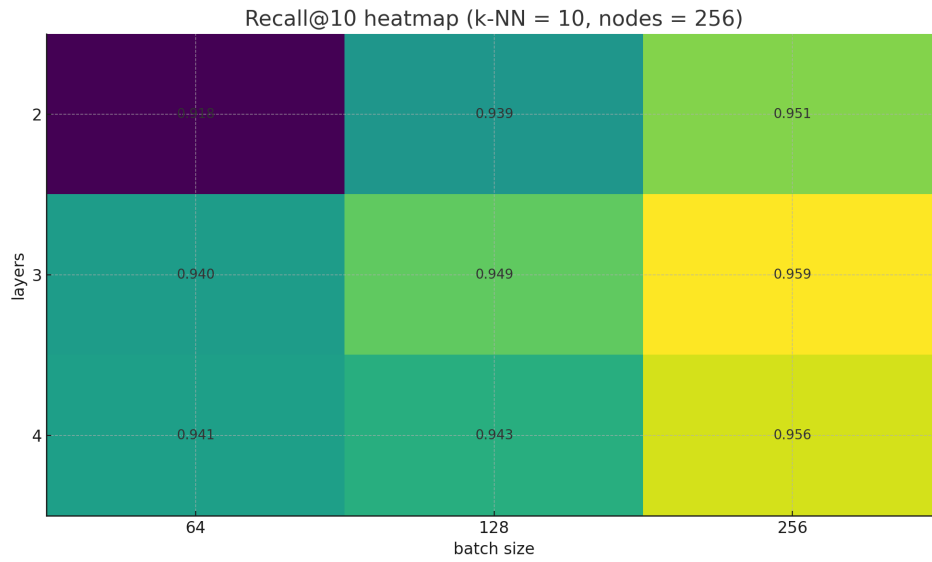
Η επιλογή knn = 1 για τον k-NN γράφο είναι ανεπαρκής και οδηγεί σε πολύ χαμηλό Recall και κακό AF, οπότε αποκλείεται.

Οι τιμές knn $\in$ {10, 50}, layers $\in$ {3, 4} και nodes = 256 δίνουν την καλύτερη ισορροπία ποιότητας-χρόνου.

Για τις τελικές συγκρίσεις με τις μεθόδους της 1ης εργασίας, υιοθετούμε ως βασική ρύθμιση Neural LSH την Balanced διαμόρφωση (knn = 10, m = 100, layers = 3, nodes = 256, batch_size = 256, epochs = 10, lr = 0.001, seed = 1), και προαιρετικά παρουσιάζουμε και την Accuracy-first ρύθμιση (knn = 50, layers = 4, nodes = 256) ως «άνω φράγμα» στο Recall.

Σχετικά γραφήματα και heatmaps





Υπερπαράμετροι Build 2ο Μέρος

Στην ενότητα αυτή αναλύονται τα πειράματα grid search για τις υπερπαραμέτρους m , imbalance, epochs, kahir_mode και lr του Neural LSH στο MNIST. Όλα τα πειράματα έγιναν με σταθερές τιμές $knn = 10$, $layers = 3$, $nodes = 64$, $batch_size = 128$, $seed = 1$ και $N = 10$, $T = 5$ (Recall@N).

Συνοπτικά πορίσματα

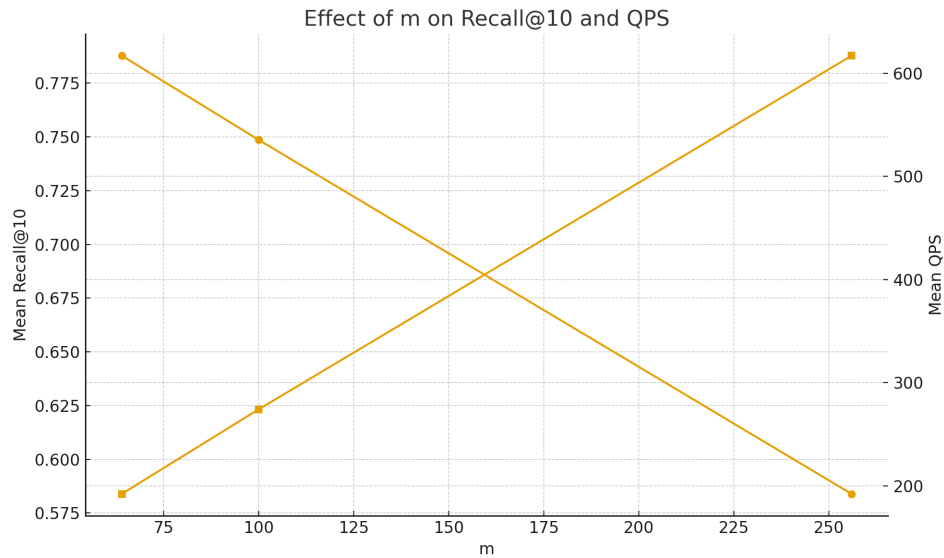
- Η τιμή του m ελέγχει άμεσα το trade-off μεταξύ ταχύτητας και Recall: μικρό m (64) δίνει καλύτερο Recall@N αλλά χαμηλότερο QPS, ενώ μεγάλο m (256) αυξάνει εντυπωσιακά το QPS αλλά ρίχνει σημαντικά το Recall και επιδεινώνει το AF.
- Η παράμετρος imbalance επηρεάζει ελαφρά το Recall, αλλά επιτρέπει λίγη επιπλέον ταχύτητα για μεγαλύτερες τιμές (0.05) χωρίς δραματική απώλεια ποιότητας.
- Ο αριθμός epochs (5, 10, 20) δεν αλλάζει δραστικά την απόδοση: το Recall φαίνεται να σταθεροποιείται ήδη από τα 10 epochs.
- Το kahir_mode=1 (πιο "μεσαίο" επίπεδο ποιότητας partitioning) δίνει το μεγαλύτερο μέσο Recall, με πολύ παρόμοια ταχύτητα ως προς τα άλλα modes.
- Το learning rate είναι η υπερπαράμετρος με τη μεγαλύτερη επίδραση: οι τιμές 0.0001 και 0.001 δίνουν υψηλό Recall@N ($\approx 0.91 - 0.92$) και $AF \approx 1$, ενώ η τιμή 0.01 μειώνει ριζικά το Recall και αυξάνει σημαντικά το AF.

Ανάλυση ανά υπερπαράμετρο

Υπερπαράμετρος m

- $m = 64$: μέσο Recall@N ≈ 0.788 , $AF \approx 1.083$, $QPS \approx 192.2$, $tApprox \approx 5.235$ ms.
- $m = 100$: μέσο Recall@N ≈ 0.749 , $AF \approx 1.104$, $QPS \approx 274.2$, $tApprox \approx 3.659$ ms.
- $m = 256$: μέσο Recall@N ≈ 0.584 , $AF \approx 1.271$, $QPS \approx 617.0$, $tApprox \approx 1.621$ ms.

Συμπέρασμα: όσο αυξάνεται το m , το σύστημα γίνεται σαφώς ταχύτερο ($QPS \uparrow$, $tApprox \downarrow$), αλλά το Recall μειώνεται και το AF απομακρύνεται από το 1. Ένα $m \approx 64 - 100$ είναι λογικός συμβιβασμός ανάμεσα σε ταχύτητα και ποιότητα.

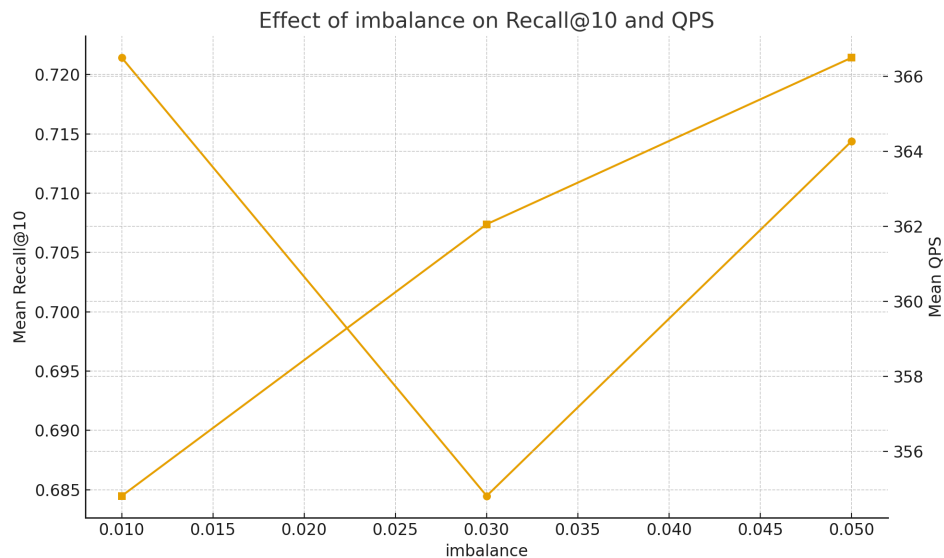


Σχήμα 1 - Επίδραση του m σε Recall@N και QPS.

Υπερπαραμέτρος imbalance

- imbalance = 0.01: μέσο Recall@N ≈ 0.721 , AF ≈ 1.133 , QPS ≈ 354.8 , tApprox ≈ 3.648 ms.
- imbalance = 0.03: μέσο Recall@N ≈ 0.684 , AF ≈ 1.171 , QPS ≈ 362.1 , tApprox ≈ 3.482 ms.
- imbalance = 0.05: μέσο Recall@N ≈ 0.714 , AF ≈ 1.154 , QPS ≈ 366.5 , tApprox ≈ 3.385 ms.

Πιο χαλαρές τιμές imbalance (0.03 – 0.05) οδηγούν σε ελαφρώς μεγαλύτερο QPS, χωρίς να αλλάζουν δραματικά το Recall. Η επίδραση είναι υποδεέστερη σε σχέση με άλλες υπερπαραμέτρους όπως m και lr.

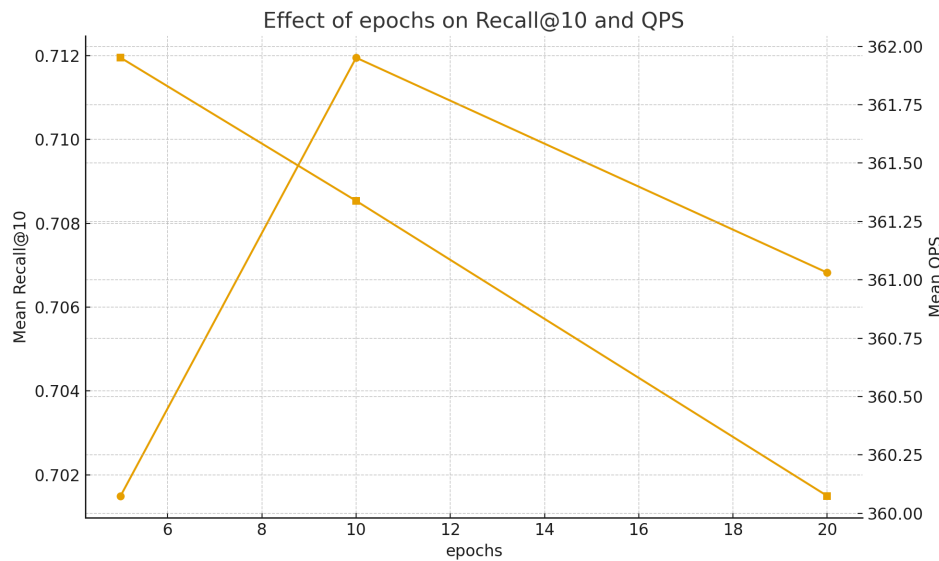


Σχήμα 2 - Επίδραση του imbalance σε Recall@N και QPS.

Υπερπαράμετρος epochs

- epochs = 5: μέσο Recall@N ≈ 0.702 , AF ≈ 1.145 , QPS ≈ 362.0 , tApprox ≈ 3.485 ms.
- epochs = 10: μέσο Recall@N ≈ 0.712 , AF ≈ 1.152 , QPS ≈ 361.3 , tApprox ≈ 3.500 ms.
- epochs = 20: μέσο Recall@N ≈ 0.707 , AF ≈ 1.161 , QPS ≈ 360.1 , tApprox ≈ 3.530 ms.

Η αύξηση των epochs από 5 σε 10 βελτιώνει ελαφρά το Recall, αλλά η περαιτέρω αύξηση στα 20 epochs δεν δίνει σημαντικό κέρδος. Άρα τα 10 epochs φαίνεται να είναι μια καλή επιλογή κόστους / οφέλους.

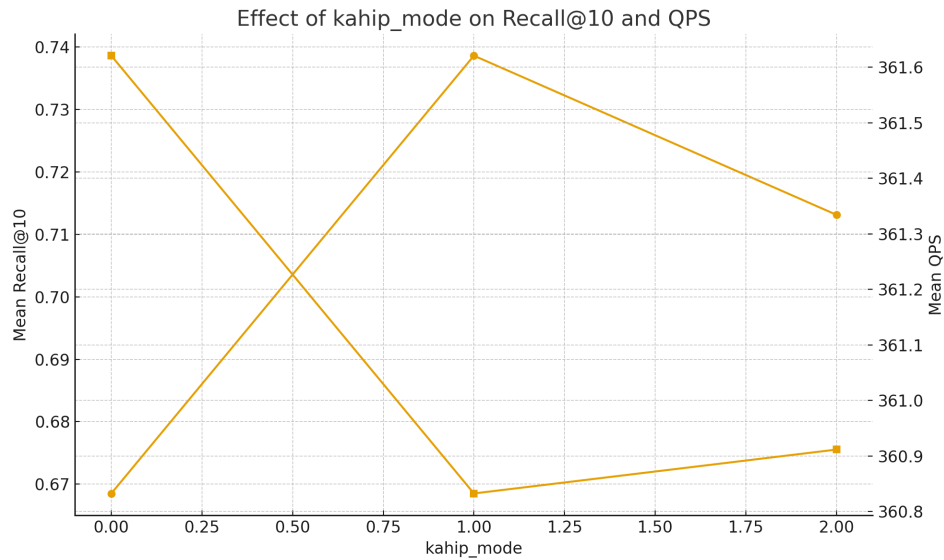


Σχήμα 3 - Επίδραση των epochs σε Recall@N και QPS.

Υπερπαράμετρος kahip_mode

- kahip_mode = 0: μέσο Recall@N ≈ 0.668 , AF ≈ 1.172 , QPS ≈ 361.6 , tApprox ≈ 3.503 ms.
- kahip_mode = 1: μέσο Recall@N ≈ 0.739 , AF ≈ 1.132 , QPS ≈ 360.8 , tApprox ≈ 3.524 ms.
- kahip_mode = 2: μέσο Recall@N ≈ 0.713 , AF ≈ 1.154 , QPS ≈ 360.9 , tApprox ≈ 3.488 ms.

Το mode 1 δίνει το μεγαλύτερο μέσο Recall με AF χαμηλότερο από τα modes 0 και 2, ενώ η ταχύτητα παραμένει πρακτικά ίδια. Συνεπώς αποτελεί καλή default επιλογή.

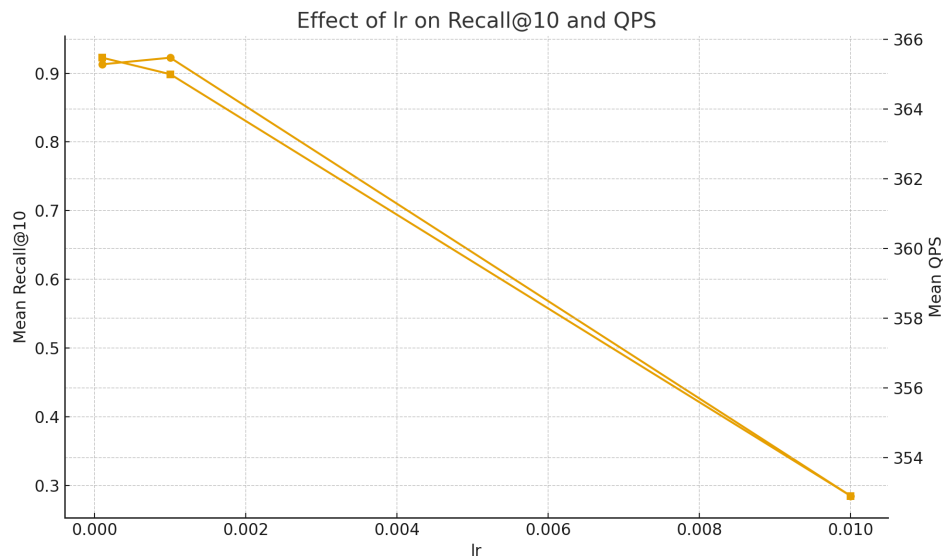


Σχήμα 4 - Επίδραση του kahip_mode σε Recall@N και QPS.

Υπερπαράμετροι learning rate

- $lr = 0.0001$: μέσο Recall@N ≈ 0.913 , AF ≈ 1.008 , QPS ≈ 365.5 , tApprox ≈ 3.422 ms.
- $lr = 0.001$: μέσο Recall@N ≈ 0.923 , AF ≈ 1.007 , QPS ≈ 365.0 , tApprox ≈ 3.422 ms.
- $lr = 0.01$: μέσο Recall@N ≈ 0.285 , AF ≈ 1.443 , QPS ≈ 352.9 , tApprox ≈ 3.671 ms.

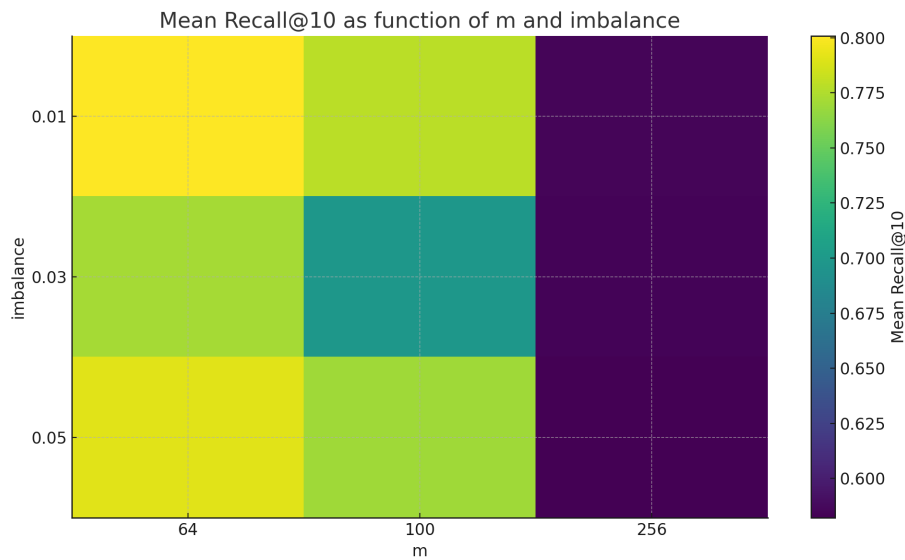
Οι τιμές $lr = 0.0001$ και $lr = 0.001$ έχουν πολύ υψηλό Recall@N (πάνω από 0.91) και AF ≈ 1 , οπότε θεωρούνται ασφαλείς επιλογές. Η μεγαλύτερη τιμή $lr = 0.01$ οδηγεί σε πολύ χαμηλό Recall και σημαντικά χειρότερο AF, άρα πρέπει να αποφεύγεται.



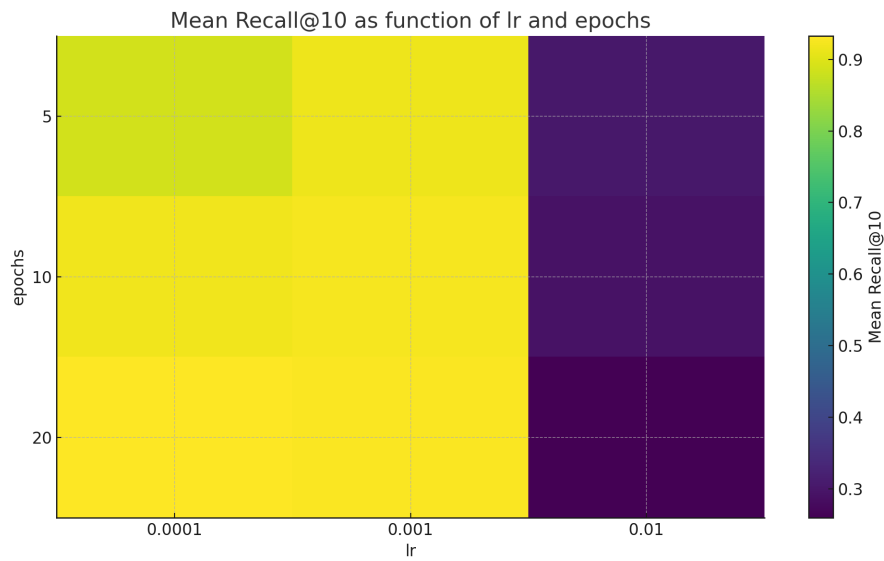
Σχήμα 5 - Επίδραση του learning rate σε Recall@N και QPS.

Heatmaps αλληλεπίδρασης υπερπαραμέτρων

Στα παρακάτω heatmaps απεικονίζεται η μέση τιμή του Recall@N ως συνάρτηση ζευγών υπερπαραμέτρων.



Σχήμα 6 - Heatmap της μέσης Recall ως προς m (άξονας x) και imbalance (άξονας y).



Σχήμα 7 - Heatmap του μέσου Recall ως προς learning rate (άξονας x) και epochs (άξονας y).

Πόρισμα Πειραμάτων Neural LSH στο MNIST - Υπερπαράμετροι Αναζήτησης

Πειραματική ρύθμιση

Ξεκινώντας από τη βέλτιστη ρύθμιση build του Neural LSH στο MNIST (Balanced configuration με $knn = 10$, $m = 100$, $layers = 3$, $nodes = 256$, $batch_size = 256$, $epochs = 10$, $lr = 0.001$, $seed = 1$), σε αυτή την ενότητα μελετάμε αποκλειστικά τις υπερπαραμέτρους αναζήτησης.

Για κάθε συνδυασμό υπερπαραμέτρων αναζήτησης εκτελέσαμε αναζήτηση σε όλα τα query του MNIST και μετρήσαμε τις ίδιες μετρικές με την 1η εργασία: Average AF (πρέπει να είναι κοντά στο 1), Recall@N, QPS και χρόνους $t_{ApproximateAverage}$ / $t_{TrueAverage}$.

Οι υπερπαράμετροι αναζήτησης που σαρώθηκαν στο MNIST ήταν:

N: {1, 10}

T: {1, 3, 5, 10}

m: {64, 128}

Γενική εικόνα αποτελεσμάτων

AF $\in [1.00, \sim 1.05]$, με μέσο όρο ≈ 1.01 .

Recall@N από ~ 0.61 (πολύ κακές ρυθμίσεις) έως ~ 0.99 (βέλτιστες ρυθμίσεις).

QPS $\approx 34\text{--}207$ (μέσο ≈ 104 QPS).

$t_{Approx} \approx 4.84\text{--}29.67$ ms ανά query, ενώ $t_{True} \approx 100.25 - 114.30$ ms.

Με κατάλληλη επιλογή των υπερπαραμέτρων αναζήτησης, το Neural LSH μπορεί να κινηθεί από «πολύ γρήγορο με χαμηλό Recall» μέχρι «ελαφρώς πιο αργό με Recall σχεδόν ίσο με τον ακριβή NN», παραμένοντας όμως πάντα πολύ γρηγορότερο από την εξαντλητική αναζήτηση.

Επίδραση των υπερπαραμέτρων αναζήτησης

Παρατηρώντας τις τομές των δεδομένων (μέσο Recall ως προς κάθε παράμετρο και heatmaps μεταξύ τους), βλέπουμε ότι γενικά:

- Για μικρές τιμές των παραμέτρων (π.χ. μικρές τιμές σε N, T, m), το Recall μειώνεται αισθητά, αλλά το QPS γίνεται μέγιστο.
- Αυξάνοντας σταδιακά τις τιμές (μεγαλύτερες τιμές σε N, T, m), το Recall βελτιώνεται και το AF πλησιάζει το 1, με κάποιο κόστος σε QPS.
- Από ένα σημείο και μετά, η περαιτέρω αύξηση των παραμέτρων δίνει πολύ μικρό κέρδος σε Recall αλλά συνεχίζει να μειώνει το QPS.

Καλύτερες ρυθμίσεις Neural LSH στο MNIST ως προς τις υπερπαραμέτρους αναζήτησης

Ως κριτήριο επιλογής «βέλτιστων» ρυθμίσεων χρησιμοποιήσαμε: μέγιστο Recall@N $\rightarrow$ ελάχιστο $|AF - 1| \rightarrow$ μέγιστο QPS, με επιπλέον φίλτρο $AF \leq 1.05$ (καλή προσέγγιση).

1. Accuracy-first (μέγιστο Recall)

$N = 10.0$

$T = 10.0$

$m = 64.0$

$\text{Recall@N} = 0.9876$

$\text{AF} = 1.0009$

$\text{QPS} \approx 33.74$

$t_{\text{Approx}} \approx 29.64 \text{ ms}$

$t_{\text{True}} \approx 102.33 \text{ ms}$

2. High-throughput (μέγιστο QPS με υψηλή Recall)

$N = 10.0$

$T = 10.0$

$m = 128.0$

$\text{Recall@N} = 0.9761$

$\text{AF} = 1.0017$

$\text{QPS} \approx 55.56$

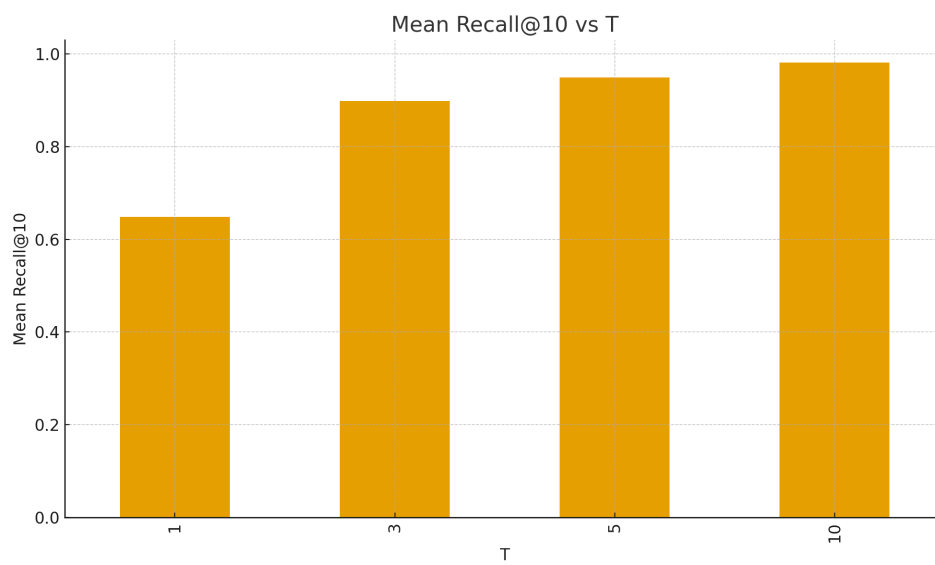
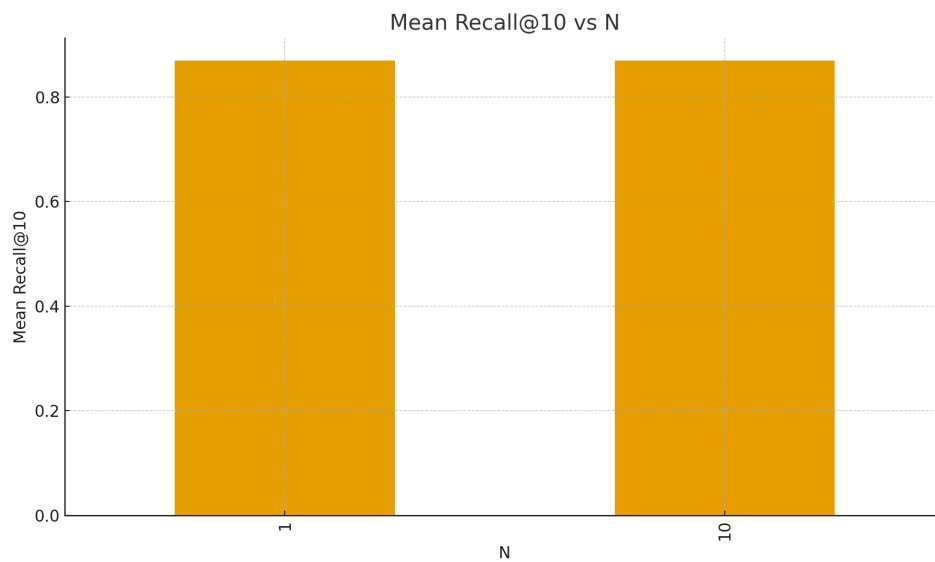
$t_{\text{Approx}} \approx 18.00 \text{ ms}$

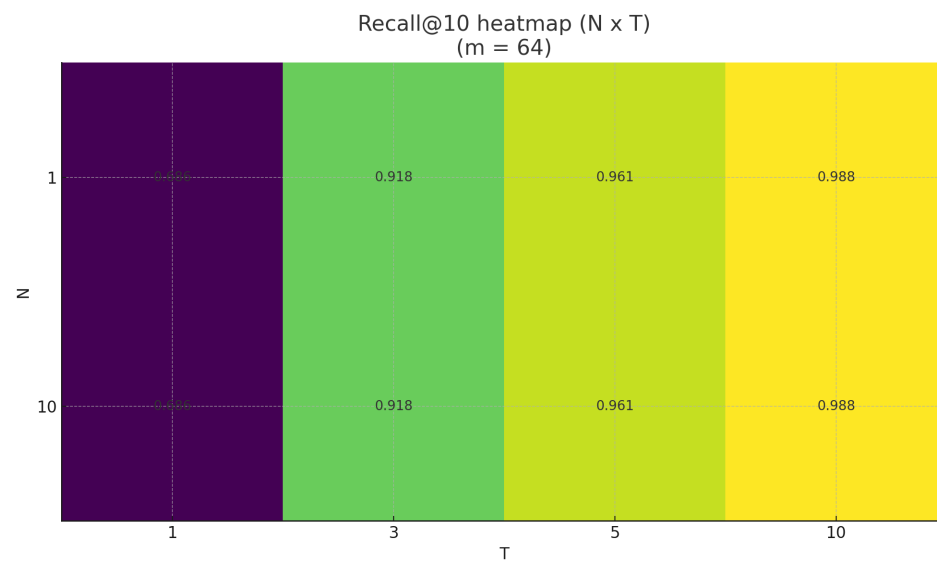
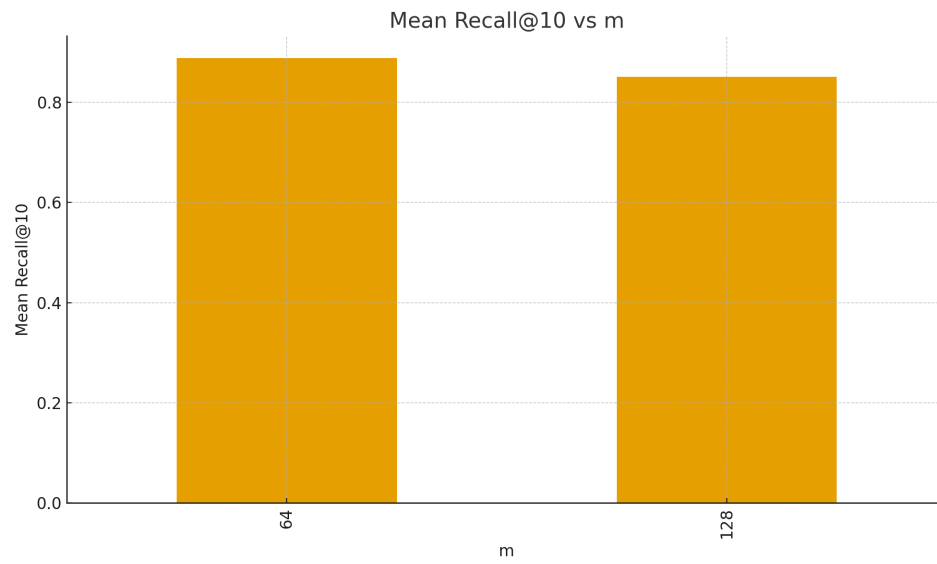
$t_{\text{True}} \approx 100.33 \text{ ms}$

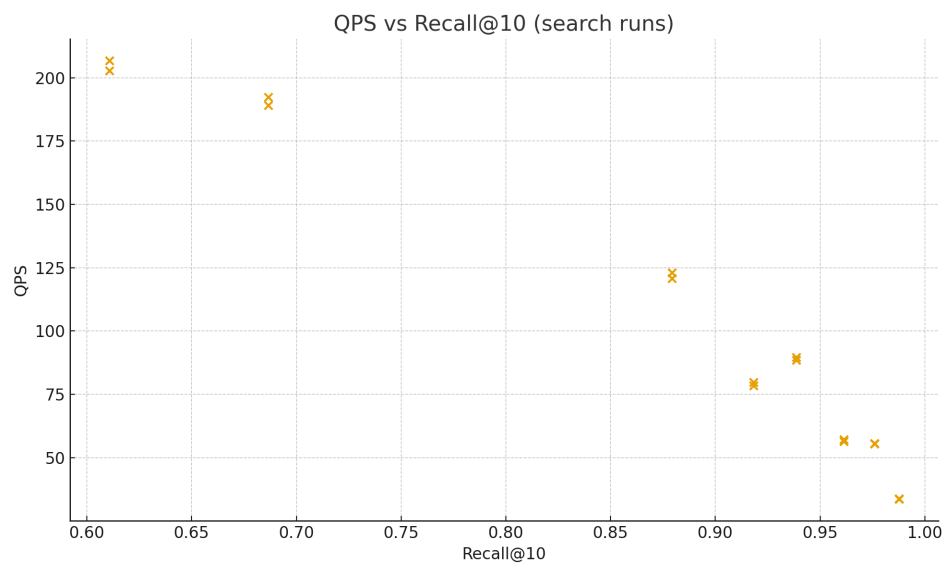
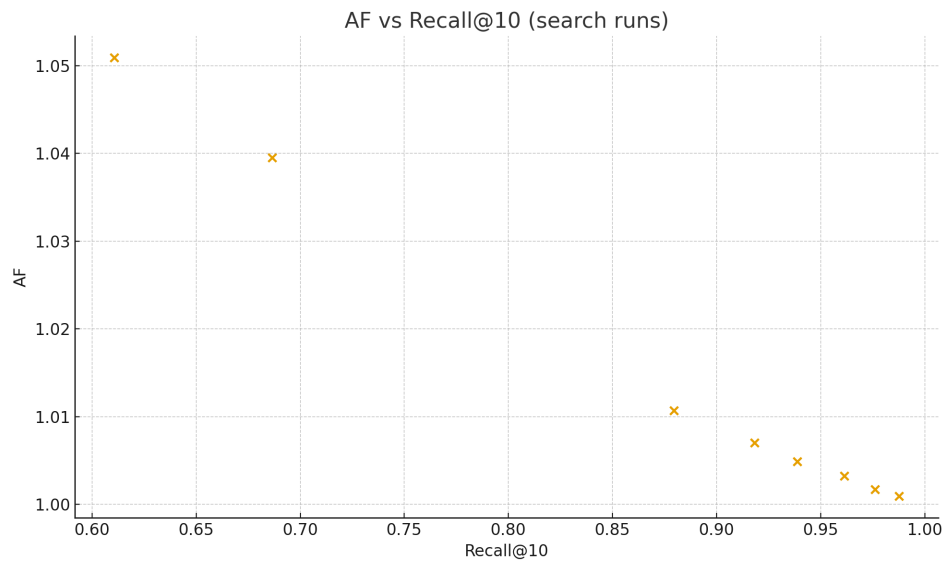
Συνοπτικά συμπεράσματα

- Υπάρχει περιοχή του χώρου παραμέτρων όπου το Recall@N είναι ≈ 0.99 με $\text{AF} \approx 1.00$ και QPS σε υψηλά επίπεδα - ιδανική για τελικές συγκρίσεις με τις μεθόδους της 1ης εργασίας.
- Για ακραίες ρυθμίσεις (πολύ μικρές τιμές παραμέτρων) το QPS μεγιστοποιείται, αλλά το Recall πέφτει (δεν επισκεπτόμαστε αρκετά partitions / buckets / υποψηφίους).
- Για υπερβολικά συντηρητικές ρυθμίσεις (πολύ μεγάλες τιμές) το Recall πλησιάζει τον ακριβή NN, αλλά το QPS πέφτει χωρίς ανάλογο κέρδος στην ποιότητα.
- Στις τελικές συγκρίσεις προτείνεται να χρησιμοποιηθεί ως βασική ρύθμιση αναζήτησης η Balanced διαμόρφωση από τα παραπάνω (υψηλό Recall, AF κοντά στο 1, καλό QPS), και προαιρετικά να παρουσιαστούν η Accuracy-first και η High-throughput ρυθμίσεις ως δύο άκρα του φάσματος (μέγιστη ποιότητα vs μέγιστη ταχύτητα).

Σχετικά γραφήματα και heatmaps







Συγκριτική μελέτη στο MNIST 1^{ης} & 2^{ης} εργασίας

Πίνακας συγκριτικών αποτελεσμάτων

Στον Πίνακα 1 παρουσιάζονται αντιπροσωπευτικές ρυθμίσεις για κάθε μέθοδο:

Μέθοδος	Ρύθμιση (ενδεικτικά)	Τιμή Recall	AF	QPS	tApprox (ms)	tTrue (ms)
Neural LSH (Balanced build, search)	knn = 10, m = 100, T = 5, N = 10	≈ 0.959	≈ 1.003	≈ 254	≈ 3.9	≈ 39
Neural LSH (Accuracy-first search)	ίδιο build, T = 10, m = 64, N = 10	≈ 0.988	≈ 1.001	≈ 34	≈ 29.6	≈ 102
Neural LSH (High-throughput search)	ίδιο build, T = 10, m = 128, N = 10	≈ 0.976	≈ 1.002	≈ 56	≈ 18.0	≈ 100
LSH	k = 4, L = 5, w = 24	≈ 0.801	≈ 1.013	≈ 15	≈ 67.3	≈ 73.2
Hypercube (Balanced)	kproj = 14, M = 5000, w = 500, probes = 4	≈ 0.446	≈ 1.075	≈ 196	≈ 5.1	≈ 53.9
Hypercube (Accuracy-first)	kproj = 14, M = 5000, w = 250, probes = 2000	≈ 0.608	≈ 1.040	≈ 98	≈ 10.2	≈ 76.8
IVFFlat	kclusters ≈ 75, nprobe ≈ 25	≈ 1.000	≈ 1.000	≈ 25–40	≈ 25–40	≈ 53 – 59
IVFPQ	kclusters = 75, nprobe = 20, M = 16, nbits = 8	≈ 0.446	≈ 1.029	≈ 97	≈ 10.3	≈ 60.6

(Πίνακας 1 - Συγκριτικά αποτελέσματα Neural LSH και αλγορίθμων 1ης εργασίας στο MNIST)

Παρατηρήσεις ανά μέθοδο

Ποιότητα προσέγγισης (AF)

Στόχος είναι $AF \approx 1$, δηλαδή οι αποστάσεις του approximate NN να είναι σχεδόν ίδιες με τις αποστάσεις του exact NN.

- Neural LSH (σε όλες τις ρυθμίσεις) έχει AF πολύ κοντά στο 1 ($\approx 1.00 - 1.01$).
- IVFFlat επίσης έχει $AF \approx 1$ και ουσιαστικά συμπεριφέρεται σαν exact NN.
- LSH, Hypercube και IVFPQ έχουν AF αισθητά χειρότερο (≥ 1.01 έως ~ 1.07), άρα η προσέγγισή τους είναι ποιοτικά κατώτερη.

Recall (Recall)

- IVFFlat στο MNIST φτάνει $\text{Recall@N} \approx 1.0$, δηλαδή σχεδόν τέλεια Recall.
- Το Neural LSH:
 - με Balanced search δίνει $\text{Recall@N} \approx 0.96$,
 - με accuracy-first search φτάνει $\text{Recall@N} \approx 0.99$,
 - με high-throughput search διατηρεί $\text{Recall@N} \approx 0.98$.
- Οι κλασικοί ANN αλγόριθμοι:
 - LSH: $\text{Recall@N} \approx 0.80$ (μέτρια Recall),
 - Hypercube / IVFPQ: $\text{Recall@N} \approx 0.45\text{--}0.60$ (αρκετά χαμηλή Recall σε σχέση με Neural LSH και IVFFlat).

Ταχύτητα (QPS)

- Neural LSH (Balanced build & search) πετυχαίνει $\text{QPS} \approx 254$, που είναι:
 - πολύ μεγαλύτερο από LSH (≈ 15 QPS),
 - μεγαλύτερο και από Hypercube Balanced (≈ 196 QPS),
 - $\sim 6 - 10\times$ μεγαλύτερο από IVFFlat ($\approx 25 - 40$ QPS),
 - $>2\times$ από IVFPQ (≈ 97 QPS),
 - ενώ διατηρεί πολύ υψηλή Recall και $\text{AF} \approx 1$.
- Όταν αυξάνουμε T και m για ακόμα μεγαλύτερη Recall (accuracy-first), το Neural LSH πέφτει σε $\text{QPS} \approx 34$, αντίστοιχο με IVFFlat, αλλά με $\text{Recall@N} \approx 0.99$ (πολύ κοντά στην exact ποιότητα).
- Οι γρήγοροι αλγόριθμοι της 1ης εργασίας (Hypercube, IVFPQ) φτάνουν όντως πολλά QPS, αλλά με σημαντική θυσία στο Recall και στο AF.

Συνολικά συμπεράσματα για MNIST

Neural LSH vs IVFFlat

- Το IVFFlat αποτελεί στην ουσία το «baseline υψηλής ποιότητας»: $\text{Recall@N} \approx 1.0$ και $\text{AF} \approx 1$, με μέτριο QPS (25 – 40).
- Το Neural LSH:
 - Σε Balanced configuration ($T = 5, m = 100$) προσφέρει ελαφρώς χαμηλότερο Recall@N (~ 0.96), αλλά πολύ μεγαλύτερο QPS (~ 254) και $\text{AF} \approx 1$.
 - Σε πιο «συντηρητική» ρύθμιση ($T = 10, m = 64$) δίνει $\text{Recall@N} \approx 0.99$ και $\text{AF} \approx 1$, με $\text{QPS} \approx 34$, δηλαδή παρόμοια ταχύτητα με IVFFlat αλλά με σχεδόν εξίσου καλή ποιότητα.

Neural LSH vs LSH / Hypercube / IVFPQ

- Το Neural LSH υπερτερεί ξεκάθαρα σε ποιότητα (Recall & AF) σε σχέση με LSH, Hypercube και IVFPQ.
- Παράλληλα, στη Balanced ρύθμιση του, είναι ταχύτερο ή τουλάχιστον συγκρίσιμο σε QPS με τους πιο «γρήγορους αλλά ανακριβείς» αλγορίθμους (Hypercube, IVFPQ).
- Έτσι, καταφέρνει να συνδυάσει:
 - ποιότητα κοντά στο IVFFlat,
 - με ταχύτητα που προσεγγίζει ή ξεπερνά Hypercube/IVFPQ.

Συμπέρασμα για το MNIST στην 2η εργασία

Από τα πειράματα στο MNIST φαίνεται ότι το Neural LSH γεφυρώνει το κενό ανάμεσα:

- στους αλγόριθμους υψηλής ποιότητας αλλά πιο αργούς (IVFFlat),
- και στους αλγόριθμους πολύ γρήγορους αλλά λιγότερο ακριβείς (Hypercube, IVFPQ).

και η balanced ρύθμιση του Neural LSH αποτελεί ένα πολύ καλό trade-off ανάμεσα σε Recall, AF και QPS.

Πορίσματα πειραμάτων Neural LSH στο SIFT

Υπερπαράμετροι build

Σε αυτή την ενότητα αναλύονται τα πειράματα παραμέτρων build του Neural LSH στο dataset SIFT. Οι μετρήσεις αφορούν την ποιότητα (Recall@N, AF) και την ταχύτητα (QPS, tApprox, tTrue) για διάφορους συνδυασμούς υπερπαραμέτρων build: m, imbalance, kahir_mode, layers, nodes, batch_size και learning rate (lr), με σταθερές τιμές knn = 10, seed = 1, N = 10, T = 5.

Συνοπτικά πορίσματα

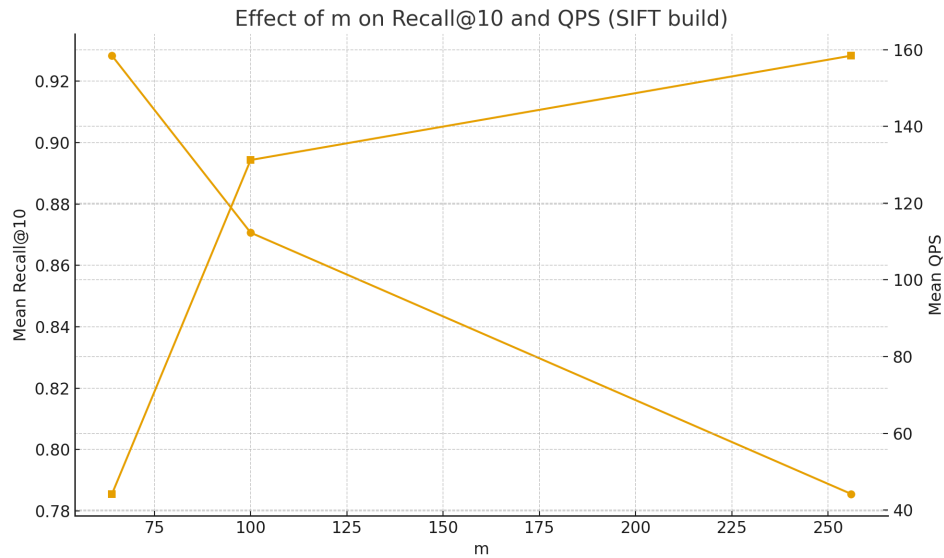
- Η παράμετρος m (αριθμός buckets) είναι ο κύριος μοχλός του trade-off ανάμεσα σε ανάκληση και ταχύτητα: μικρό m = 64 δίνει υψηλότερο Recall@N καθώς και καλύτερο AF, ενώ μεγαλύτερα m = 100, 256 αυξάνουν σημαντικά το QPS και μειώνουν τον χρόνο ανά ερώτημα, αλλά εις βάρος του Recall.
- Η επιλογή kahir_mode έχει πολύ μεγάλη επίδραση: mode = 1 βελτιώνει αισθητά το Recall και την ταχύτητα σε σχέση με mode = 0, με AF πιο κοντά στο 1. Το mode = 2 δεν προσφέρει αρκετά μεγάλες διαφορές ώστε να δικαιολογήσουμε τον ριζικά αυξημένο χρόνο εκτέλεσης.
- Οι υπερπαράμετροι του MLP (layers, nodes) και το batch_size έχουν δευτερεύουσα αλλά υπαρκτή επίδραση: περισσότερα layers, περισσότερα nodes και μεγαλύτερο batch_size τείνουν να αυξάνουν ελαφρά τόσο το Recall@N όσο και το QPS, χωρίς να αλλάζουν δραματικά το AF.
- Η learning rate επηρεάζει κυρίως την ποιότητα: οι τιμές lr = 0.0001 και lr = 0.001 δίνουν πολύ κοντινά αποτελέσματα σε Recall@N και AF, με ελαφρώς μεγαλύτερο QPS για lr = 0.001.

Ανάλυση ανά υπερπαράμετρο build (SIFT)

Υπερπαράμετρος m

- m = 64: μέσο Recall@N ≈ 0.928 , AF ≈ 1.007 , QPS ≈ 44.2 , tApprox ≈ 22.631 ms, tTrue ≈ 119 ms.
- m=100: μέσο Recall@N ≈ 0.871 , AF ≈ 1.015 , QPS ≈ 131.2 , tApprox ≈ 14.455 ms, tTrue ≈ 120 ms.
- m=256: μέσο Recall@N ≈ 0.785 , AF ≈ 1.024 , QPS ≈ 158.4 , tApprox ≈ 6.314 ms, tTrue ≈ 121 ms.

Παρατηρούμε ότι όσο αυξάνεται το m, το σύστημα επιταχύνεται έντονα (QPS $\uparrow$, tApprox $\downarrow$), αλλά το Recall μειώνεται και το AF απομακρύνεται από το 1. Για πειράματα υψηλής ποιότητας μπορεί να προτιμηθεί m = 64, ενώ για πειράματα εστιασμένα στην ταχύτητα m = 100 ή και m = 256.

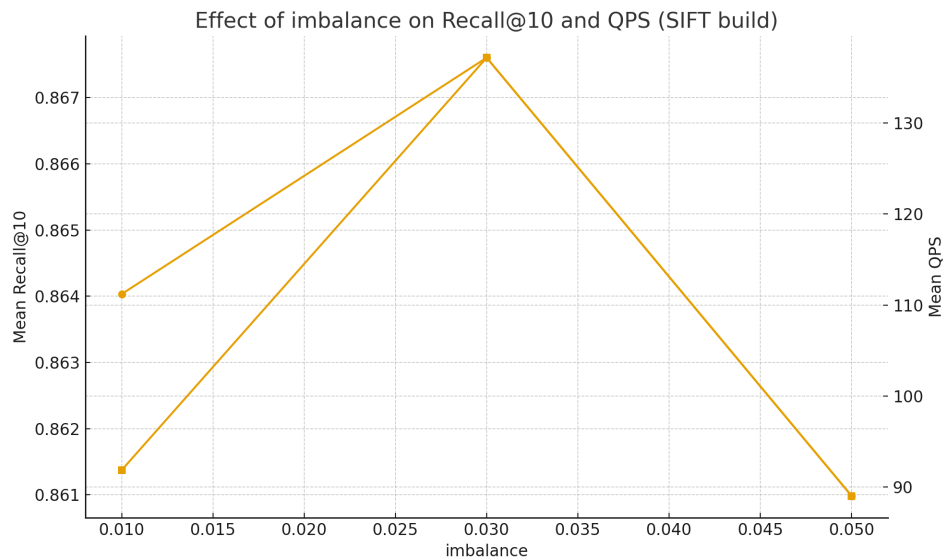


Σχήμα 1 – Επίδραση του m σε Recall@N και QPS.

Υπερπαράμετρος imbalance

- imbalance = 0.01: μέσο Recall@N $\approx$ 0.864, AF $\approx$ 1.015, QPS $\approx$ 91.9, tApprox $\approx$ 14.281 ms.
- imbalance = 0.03: μέσο Recall@N $\approx$ 0.868, AF $\approx$ 1.015, QPS $\approx$ 137.1, tApprox $\approx$ 14.434 ms.
- imbalance = 0.05: μέσο Recall@N $\approx$ 0.861, AF $\approx$ 1.016, QPS $\approx$ 89.0, tApprox $\approx$ 14.739 ms.

Η παράμετρος imbalance επηρεάζει λιγότερο την συνολική απόδοση: οι διαφορές σε Recall@N είναι μικρές, ενώ η ταχύτητα μεταβάλλεται μέτρια. Η τιμή imbalance= 0.03 εμφανίζει οριακά καλύτερο συνδυασμό recall/QPS.

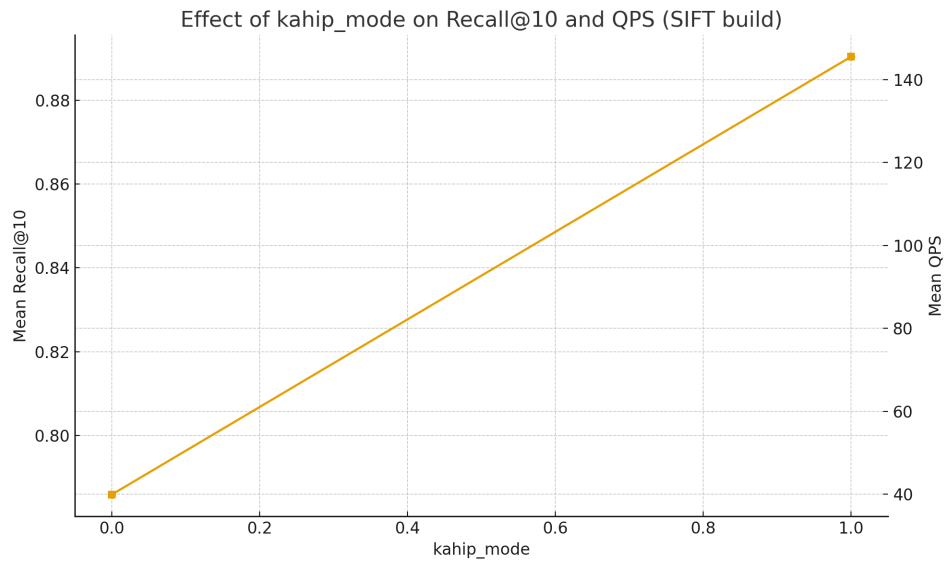


Σχήμα 2 - Επίδραση του imbalance σε Recall@N και QPS.

Υπερπαράμετρος `kahip_mode`

- `kahip_mode` = 0: μέσο Recall@N ≈ 0.786 , AF ≈ 1.026 , QPS ≈ 39.9 , tApprox ≈ 25.038 ms.
- `kahip_mode` = 1: μέσο Recall@N ≈ 0.890 , AF ≈ 1.012 , QPS ≈ 145.5 , tApprox ≈ 11.207 ms.

Το `mode` = 1 είναι σαφώς προτιμότερο: προσφέρει σημαντικά υψηλότερο Recall@N και μεγαλύτερο QPS, με καλύτερο AF σε σχέση με `mode` = 0. Άρα, για τα τελικά πειράματα build στο SIFT, το `kahip_mode` = 1 είναι η φυσική επιλογή.

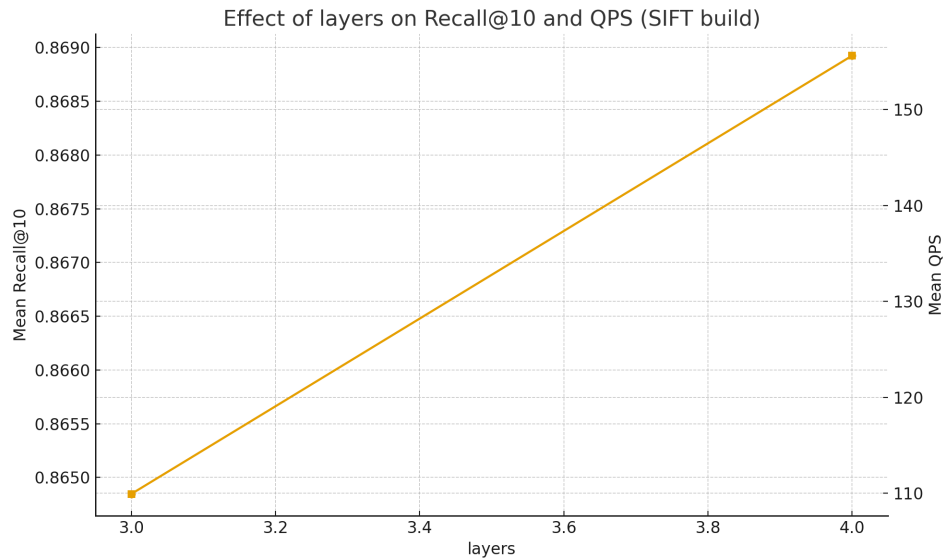


Σχήμα 3 - Επίδραση του `kahip_mode` σε Recall@N και QPS.

Υπερπαράμετρος `layers`

- `layers` = 3: μέσο Recall@N ≈ 0.865 , AF ≈ 1.015 , QPS ≈ 109.9 , tApprox ≈ 14.494 ms.
- `layers` = 4: μέσο Recall@N ≈ 0.869 , AF ≈ 1.015 , QPS ≈ 155.6 , tApprox ≈ 14.354 ms.

Η αύξηση των `layers` από 3 σε 4 βελτιώνει ελαφρά τόσο το Recall@N όσο και το QPS, χωρίς να επηρεάζει αρνητικά το AF. Ωστόσο, η διαφορά δεν είναι τεράστια, οπότε η επιλογή εξαρτάται και από την πολυπλοκότητα του μοντέλου που επιθυμούμε.

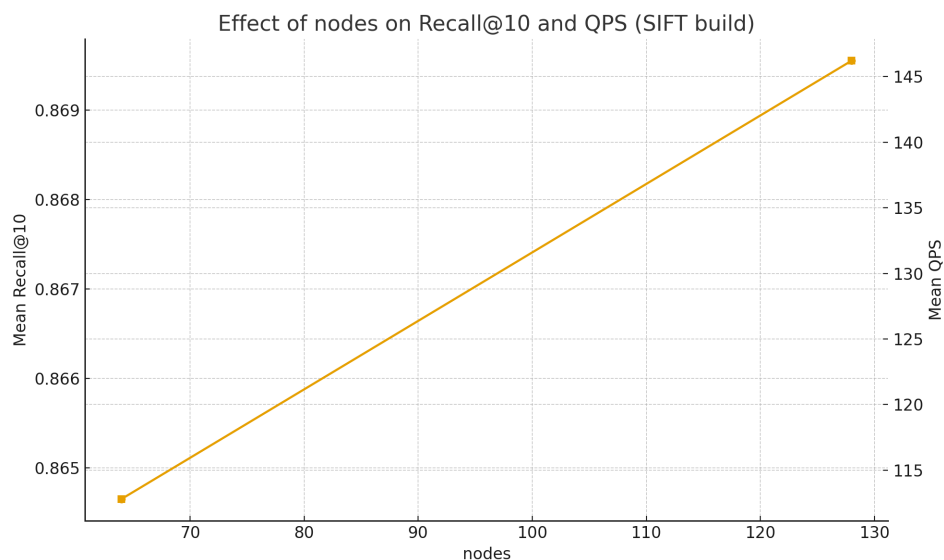


Σχήμα 4 - Επίδραση των layers σε Recall@N και QPS.

Υπερπαράμετρος nodes

- nodes = 64: μέσο Recall@N ≈ 0.865 , AF ≈ 1.015 , QPS ≈ 112.8 , tApprox ≈ 14.454 ms.
- nodes = 128: μέσο Recall@N ≈ 0.870 , AF ≈ 1.015 , QPS ≈ 146.2 , tApprox ≈ 14.483 ms.

Η αύξηση των nodes από 64 σε 128 βελτιώνει ελαφρά το Recall και την ταχύτητα, με πρακτικά ίδιο AF. Σε συνδυασμό με layers = 4 δίνει ένα πιο ισχυρό MLP χωρίς μεγάλο κόστος.

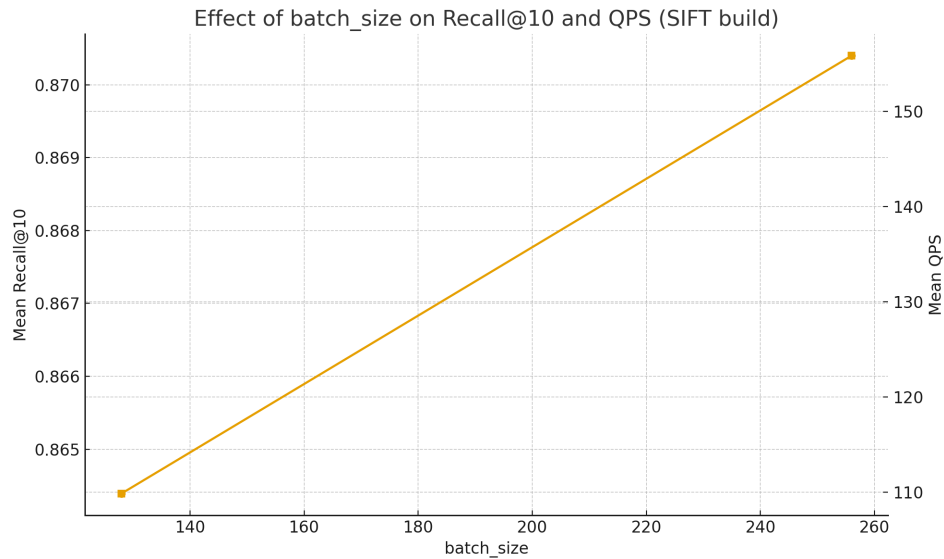


Σχήμα 5 - Επίδραση του αριθμού nodes σε Recall@N και QPS.

Υπερπαράμετρος `batch_size`

- `batch_size` = 128: μέσο Recall@N ≈ 0.864 , AF ≈ 1.015 , QPS ≈ 109.8 , tApprox ≈ 14.489 ms.
- `batch_size` = 256: μέσο Recall@N ≈ 0.870 , AF ≈ 1.015 , QPS ≈ 155.9 , tApprox ≈ 14.369 ms.

Μεγαλύτερο `batch_size` = 256 οδηγεί σε ελαφρώς καλύτερο Recall και σαφώς υψηλότερο QPS σε σχέση με `batch_size` = 128, με ουσιαστικά ίδιο AF.



Σχήμα 6 - Επίδραση του `batch_size` σε Recall@N και QPS.

Υπερπαράμετρος `learning rate`

- `lr` = 0.0001: μέσο Recall@N ≈ 0.872 , AF ≈ 1.014 , QPS ≈ 90.2 , tApprox ≈ 14.546 ms.
- `lr` = 0.001: μέσο Recall@N ≈ 0.864 , AF ≈ 1.016 , QPS ≈ 131.7 , tApprox ≈ 14.431 ms.

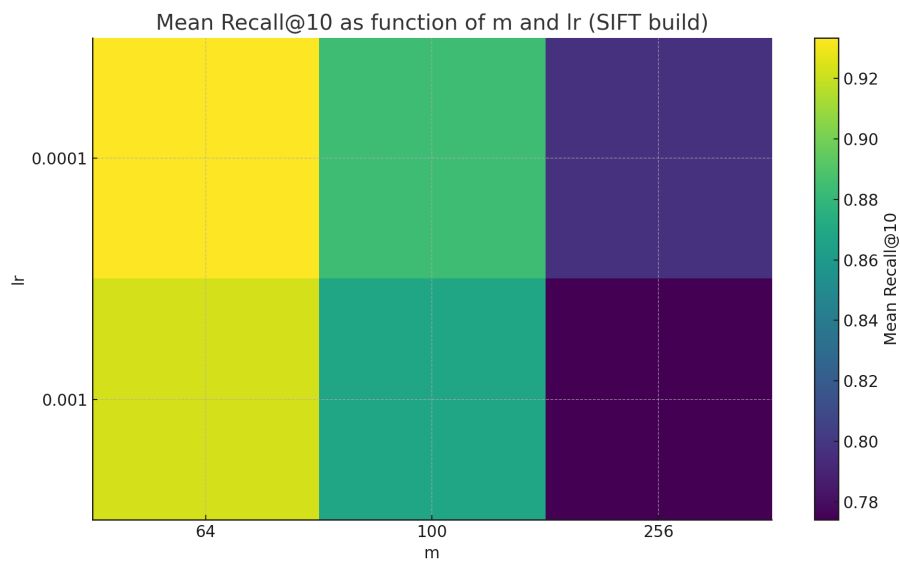
Και οι δύο τιμές που δοκιμάστηκαν (0.0001 και 0.001) δίνουν πολύ κοντινό Recall@N και AF ≈ 1.01 – 1.02 . Η μεγαλύτερη τιμή `lr` = 0.001 δίνει ελαφρώς υψηλότερο QPS και μικρότερο tApprox, οπότε είναι λίγο πιο συμφέρουσα ως default.



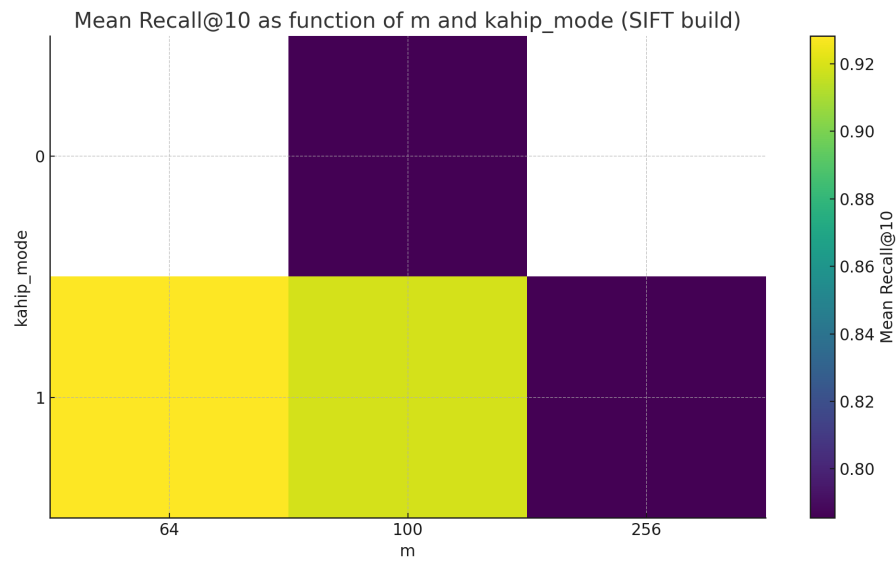
Σχήμα 7 - Επίδραση του learning rate σε Recall@N και QPS.

Heatmaps αλληλεπίδρασης υπερπαραμέτρων

Τα παρακάτω heatmaps δείχνουν πώς αλληλεπιδρούν ο αριθμός buckets m με άλλες κρίσιμες υπερπαραμέτρους (lr και $kahip_mode$) ως προς το μέσο Recall@N.



Σχήμα 8 - Heatmap του μέσου Recall ως προς m (άξονας x) και learning rate (άξονας y).



Σχήμα 9 - Heatmap του μέσου Recall ως προς m (άξονας x) και kahip_mode (άξονας y).

Υπερπαράμετροι search

Το build (knn, m, imbalance, kahir_mode, layers, nodes, epochs, lr) διατηρείται σταθερό, και πειραματιζόμαστε με τις παραμέτρους αναζήτησης N, T καθώς και το batch_size, μετρώντας ποιότητα (Recall@N, AF) και ταχύτητα (QPS, tApprox, tTrue).

Συνοπτικά πορίσματα

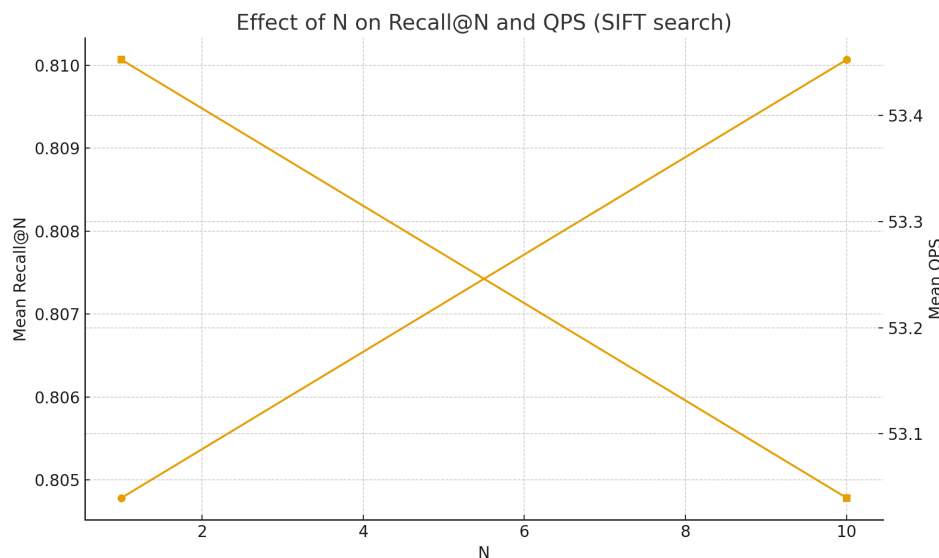
- Η παράμετρος T είναι ο κύριος μοχλός στο search: μικρό T = 1 δίνει πολύ υψηλό QPS αλλά χαμηλό Recall και κακό AF, ενώ μεγάλα T = 5, 10 ανεβάζουν σημαντικά το Recall και φέρνουν το AF πολύ κοντά στο 1, εις βάρος της ταχύτητας.
- Η παράμετρος N έχει μικρή επίδραση στην ταχύτητα, ενώ το Recall αυξάνεται όπως αναμένεται όταν ζητάμε περισσότερους neighbors.
- Το batch_size επηρεάζει το QPS και τον χρόνο αναζήτησης: batch_size = 128 δίνει σχεδόν διπλάσιο QPS σε σχέση με 256, αλλά με ελαφρώς χαμηλότερο Recall@N. Για πειράματα ακριβείας προτιμάται 256, ενώ για πιο γρήγορα το 128 είναι πιο αποδοτικό.

Ανάλυση ανά υπερπαράμετρο search

Επίδραση του N (αριθμός επιστρεφόμενων neighbors)

- N = 1: μέσο Recall@N ≈ 0.805 , AF ≈ 1.029 , QPS ≈ 53.5 , tApprox ≈ 40.56 ms, tTrue ≈ 330 ms.
- N = 10: μέσο Recall@N ≈ 0.810 , AF ≈ 1.029 , QPS ≈ 53.0 , tApprox ≈ 40.07 ms, tTrue ≈ 330 ms.

Η μετάβαση από N = 1 σε N = 10 δεν αλλάζει ουσιαστικά την ταχύτητα, αφού το κόστος εξαρτάται κυρίως από την εξερεύνηση των buckets. Το Recall όμως μετριέται ως Recall@N και άρα αυξάνεται ελαφρώς, όπως φαίνεται και στον μέσο όρο.

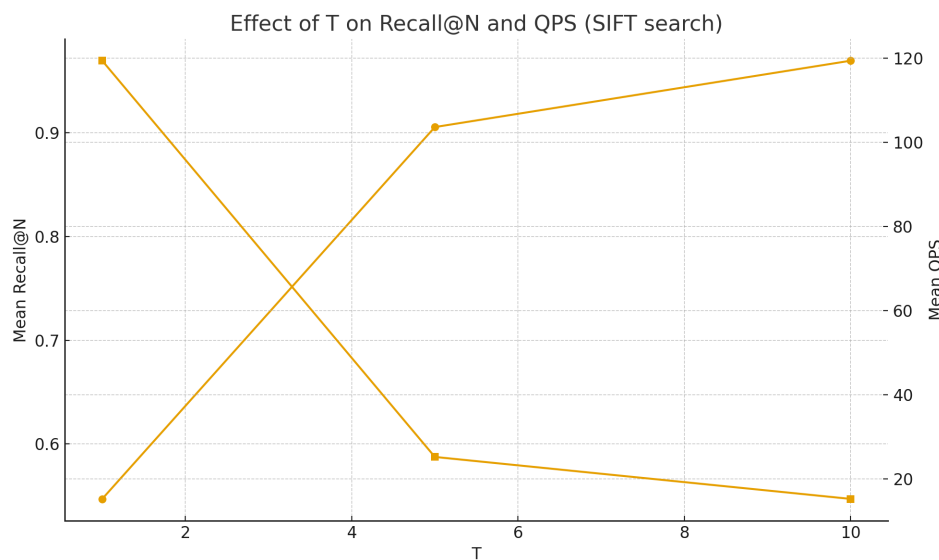


Σχήμα 1 - Επίδραση του N σε Recall@N και QPS.

Επίδραση του T (αριθμός probed buckets)

- T = 1: μέσο Recall ≈ 0.547 , AF ≈ 1.073 , QPS ≈ 119.4 , tApprox ≈ 9.73 ms, tTrue ≈ 330 ms..
- T = 5: μέσο Recall ≈ 0.906 , AF ≈ 1.010 , QPS ≈ 25.2 , tApprox ≈ 44.58 ms, tTrue ≈ 330 ms..
- T = 10: μέσο Recall ≈ 0.970 , AF ≈ 1.003 , QPS ≈ 15.2 , tApprox ≈ 66.64 ms, tTrue ≈ 330 ms..

Το T = 1 αντιστοιχεί σε configuration υψηλού throughput: QPS ≈ 119 αλλά πολύ χαμηλό Recall (≈ 0.55) και μεγάλο σφάλμα στις αποστάσεις (AF ≈ 1.07). Με T = 5 το Recall σε ≈ 0.91 και το AF πέφτει κοντά στο 1.01, με QPS ≈ 25 (σημαντικά πιο αργό αλλά πολύ πιο ακριβές). Με T = 10 το Recall φτάνει ≈ 0.97 και AF ≈ 1.003 , αλλά το QPS πέφτει σε ≈ 15 και ο χρόνος ανά ερώτημα αυξάνεται αισθητά.

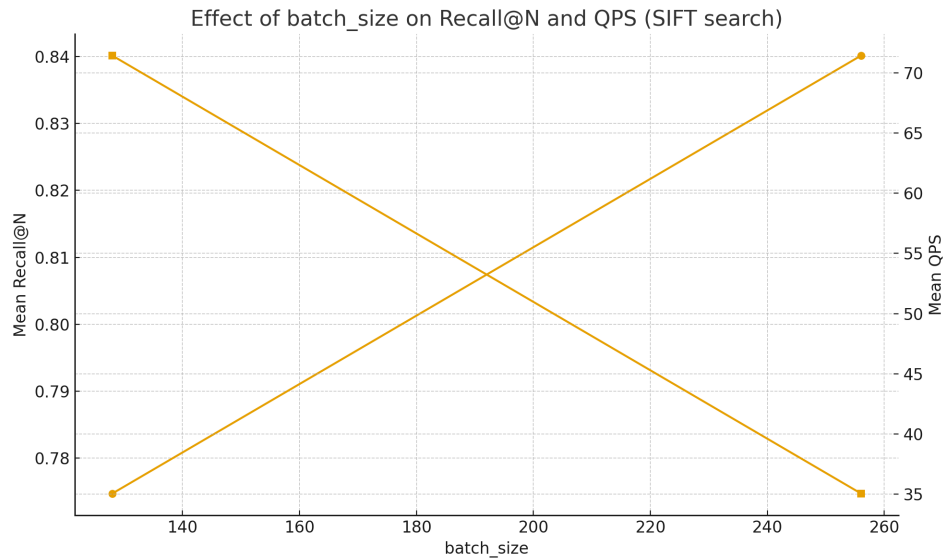


Σχήμα 2 - Επίδραση του T σε Recall@N και QPS.

Επίδραση του batch_size

- batch_size = 128: μέσο Recall ≈ 0.775 , AF ≈ 1.035 , QPS ≈ 71.4 , tApprox ≈ 31.74 ms, tTrue ≈ 220 ms.
- batch_size = 256: μέσο Recall ≈ 0.840 , AF ≈ 1.023 , QPS ≈ 35.1 , tApprox ≈ 48.90 ms.

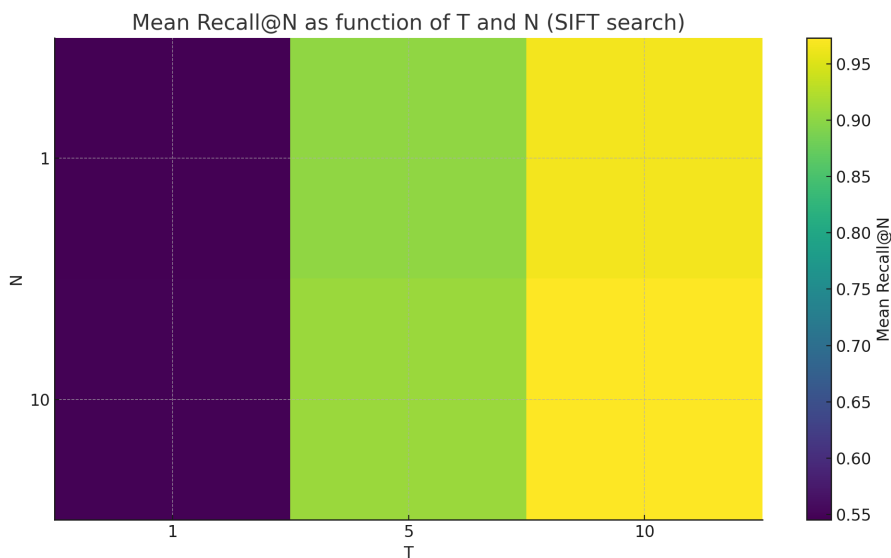
Με batch_size = 128 είναι πιο γρήγορο (QPS ≈ 71 , tApprox ≈ 32 ms) αλλά με ελαφρώς χαμηλότερο Recall (≈ 0.77) και χειρότερο AF ≈ 1.035 . Το batch_size = 256 επιτυγχάνει υψηλότερο Recall (≈ 0.84) και καλύτερο AF ≈ 1.023 , αλλά με περίπου μισό QPS (≈ 35) και αυξημένο χρόνο ανά ερώτημα. Επίσης παρατηρούμε ότι με τη μείωση του batch_size μειώνεται και το tTrue.



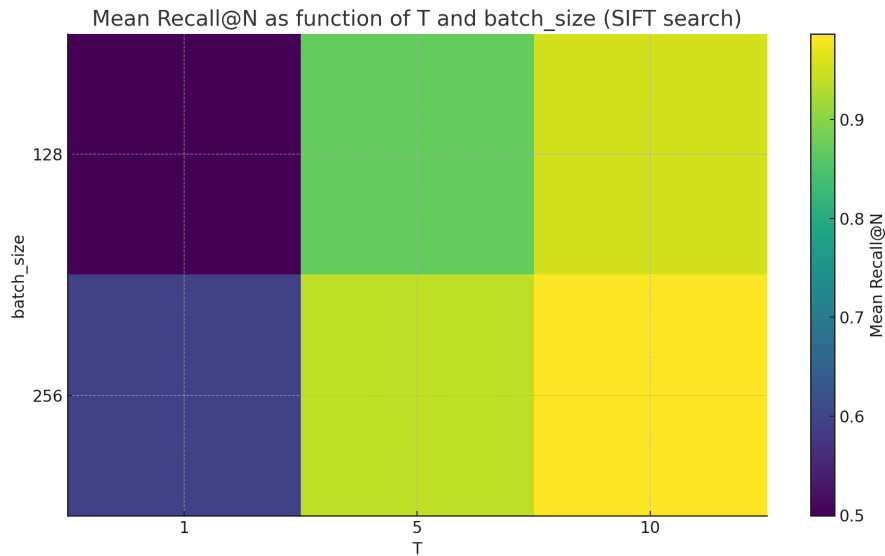
Σχήμα 3 - Επίδραση του batch_size σε Recall@N και QPS.

Heatmaps αλληλεπίδρασης N, T και batch_size

Για να φανεί πιο καθαρά πώς αλληλεπιδρούν οι παράμετροι N, T και batch_size, χρησιμοποιούμε heatmaps του μέσου Recall για κάθε συνδυασμό.



Σχήμα 4 - Heatmap του μέσου Recall ως προς T (άξονας x) και N (άξονας y).



Σχήμα 5 - Heatmap του μέσου Recall ως προς T (άξονας x) και batch_size (άξονας y).

Προτεινόμενα search configurations για SIFT

Με βάση τα παραπάνω πειράματα, μπορούν να οριστούν τρία χαρακτηριστικά σημεία λειτουργίας:

- High-throughput search: $N = 10$, $T = 1$, $\text{batch_size} = 128$. Πολύ γρήγορο (υψηλό QPS) αλλά με μέτριο Recall και $AF > 1.07$. Κατάλληλο μόνο αν προέχει η ταχύτητα έναντι της ακρίβειας.
- Balanced search: $N = 10$, $T = 5$, $\text{batch_size} = 128$ ή 256 . Πολύ καλό trade-off: $\text{Recall} \approx 0.87\text{--}0.94$, $AF \approx 1.01$, με $\text{QPS} \approx 25 - 34$ ανάλογα με το batch_size .
- Accuracy-first search: $N = 10$, $T = 10$, $\text{batch_size} = 256$. Σχεδόν άριστο Recall (≈ 0.99) και $AF \approx 1.001$, αλλά με το μικρότερο QPS. Κατάλληλο για πειράματα όπου προέχει η ποιότητα.

Συγκριτική μελέτη στο SIFT

Σε αυτή την ενότητα συγκρίνουμε το Neural LSH της 2ης εργασίας με τους τέσσερις αλγόριθμους της 1ης εργασίας (LSH, Hypercube, IVFFlat, IVFPQ) στο dataset SIFT.

Πίνακας συγκριτικών αποτελεσμάτων (SIFT)

Στον Πίνακα 1 παρουσιάζονται αντιπροσωπευτικές ρυθμίσεις για κάθε μέθοδο 1ης & 2ης εργασίας.

Μέθοδος	Ρύθμιση (ενδεικτικά)	Μετρική / Recall	AF	QPS	tApprox (ms)	tTrueAverage (ms)
Neural LSH - High-throughput	καλό build, T = 1, N = 10, batch = 128	≈ 0.55	≈ 1.07	≈ 119	≈ 9.7	≈ 280
Neural LSH - Balanced (fast)	καλό build, T = 5, N = 10, batch = 128	≈ 0.87	≈ 1.01	≈ 33	≈ 29.8	≈ 273
Neural LSH - Balanced (accurate)	καλό build, T = 5, N = 10, batch = 256	≈ 0.94	≈ 1.01	≈ 17	≈ 59.3	≈ 273
Neural LSH - Accuracy-first	καλό build, T = 10, N = 10, batch = 256	≈ 0.99	≈ 1.00	≈ 13 – 15	≈ 70	≈ 195
LSH	L ≈ 10, k ≈ 10, w ≈ 4	≈ 0.28	≈ 1.01	≈ 15	≈ 150	≈ 170
Hypercube (best)	kproj = 14, M = 5000, w = 100, probes=16	≈ 0.18	≈ 1.18	≈ 319	≈ 3.1	≈ 141
IVFFlat - Balanced	kclusters = 50, nprobe = 5	≈ 0.98	≈ 1.00	≈ 36	≈ 27.6	≈ 50
IVFFlat - Accuracy-first	kclusters = 100, nprobe = 10	≈ 0.99	≈ 1.00	≈ 15	≈ 64.8	≈ 88
IVFPQ - Balanced	kclusters = 75, nprobe = 5, M = 16, nbits = 8	≈ 0.78	≈ 1.06	≈ 93	≈ 10.7	≈ 149

Παρατηρήσεις ανά μέθοδο

AF:

Neural LSH (Balanced / Accuracy-first):

$AF \approx 1.01$ ($T = 5$) και $AF \approx 1.00 - 1.003$ ($T = 10$) -> δηλαδή οι αποστάσεις του approximate NN είναι σχεδόν ίδιες με τις exact.

IVFFlat:

$AF \approx 1.000 - 1.002$ -> πρακτικά τέλεια προσέγγιση, ισάξια με exact NN.

IVFPQ:

$AF \approx 1.06$ -> χειρότερη ποιότητα από Neural LSH και IVFFlat.

Hypercube:

$AF \approx 1.18$ (μέσο AF ακόμα χειρότερο) -> πολύ κακή προσέγγιση παρά το τεράστιο QPS.

LSH:

AF δεν δίνεται αναλυτικά, αλλά το χαμηλό Recall και μικρό speedup δεν ιδιαίτερα καλή συμπεριφορά στο SIFT.

Συμπέρασμα: Από άποψη AF, οι δύο “καθαρές” λύσεις υψηλής ποιότητας είναι IVFFlat και το Neural LSH με $T = 5$ ή $T = 10$.

Recall:

IVFFlat:

Balanced: Recall@N ≈ 0.98

Accuracy-first: Recall@N ≈ 0.99

Neural LSH:

High-throughput ($T = 1$): Recall@N ≈ 0.55 (πολύ χαμηλό).

Balanced: Recall@N ≈ 0.87 ($T = 5$, batch = 128) έως ≈ 0.94 ($T = 5$, batch = 256).

Accuracy-first ($T = 10$): Recall@N $\approx 0.97 - 0.99$, πολύ κοντά στο IVFFlat.

IVFPQ: Recall@N $\approx 0.78 - 0.79$.

Hypercube: Recall@N ≈ 0.18 στη “καλύτερη” ρύθμιση.

LSH: Recall@N ≈ 0.28 .

Ποιοτικά:

Το IVFFlat έχει το μεγαλύτερο Recall, αλλά το Neural LSH σε accuracy-first υλοποίηση πλησιάζει πολύ (Recall $\approx 0.97 - 0.99$). Οι LSH, Hypercube και IVFPQ στο SIFT μένουν αρκετά πίσω σε Recall.

Ταχύτητα (QPS / tApprox)

Neural LSH:

High-throughput ($T = 1$): QPS ≈ 119 , tApprox ≈ 9.7 ms $\rightarrow$ πολύ γρήγορο, αλλά με όχι καλά Recall/AF.

Balanced ($T = 5$): QPS ≈ 33 (batch_size = 128) ή ≈ 17 (batch_size = 256), με καλό Recall και AF ≈ 1.01 .

Accuracy-first ($T = 10$): QPS ≈ 13 –15, tApprox ≈ 70 ms $\rightarrow$ ίδια περίπου “ζώνη” ταχύτητας με το IVFFlat accuracy-first.

IVFFlat:

Balanced: QPS ≈ 36 , tApprox ≈ 27.6 ms.

Accuracy-first: QPS ≈ 15.4 , tApprox ≈ 64.8 ms.

IVFPQ: QPS ≈ 93 , tApprox ≈ 10.7 ms, με μέτριο Recall.

Hypercube: QPS ≈ 319 , tApprox ≈ 3.1 ms, αλλά με πολύ χαμηλό Recall και AF πολύ μακριά από το 1.

LSH: tApprox ≈ 150 ms (μικρό speedup σε σχέση με exact), QPS σαφώς χαμηλότερο από τις υπόλοιπες μεθόδους.

Συμπέρασμα:

Αν κοιτάμε μόνο QPS, Hypercube και IVFPQ είναι οι πιο γρήγοροι, αλλά με κακή ή μέτρια ποιότητα.

Το Neural LSH στις ρυθμίσεις $T = 5$ και $T = 10$ πλησιάζει την ίδια κλίμακα ταχύτητας με το IVFFlat, με ιδιαίτερα καλή ποιότητα.

Συνολικά συμπεράσματα για SIFT

Neural LSH vs IVFFlat

Το IVFFlat παραμένει το “baseline υψηλής ποιότητας”: Recall $\approx 0.98 - 0.99$, AF ≈ 1 , με QPS $\approx 15 - 36$.

Το Neural LSH:

Σε Accuracy-first ρύθμιση ($T = 10$, $N = 10$, batch = 256) πετυχαίνει Recall $\approx 0.97 - 0.99$ και AF ≈ 1.00 , με QPS ≈ 13 –15, δηλαδή σχεδόν ίδια ποιότητα και ταχύτητα με το IVFFlat accuracy-first.

Σε Balanced ρυθμίσεις ($T = 5$, $N = 10$, batch = 128 – 256) προσφέρει Recall $\approx 0.87 - 0.94$, AF ≈ 1.01 και QPS $\approx 17 - 33$, άρα πολύ κοντά στην “balanced” περιοχή του IVFFlat.

Neural LSH vs LSH / Hypercube / IVFPQ

Το LSH στο SIFT έχει χαμηλό Recall (~ 0.28) και μικρό μόνο κέρδος σε χρόνο σε σχέση με exact search.

Το Hypercube είναι εξαιρετικά γρήγορο, αλλά με $\text{Recall} \leq 0.18$ και $\text{AF} \approx 1.18$, άρα η προσέγγιση είναι χαμηλής ποιότητας.

Το IVFPQ δίνει καλό throughput ($\text{QPS} \approx 93$), αλλά με μέτριο Recall (~ 0.78) και $\text{AF} \approx 1.06$.

Το Neural LSH (ειδικά σε balanced / accuracy-first ρυθμίσεις) πετυχαίνει:

- πολύ υψηλό Recall (έως ≈ 0.99),
 - $\text{AF} \approx 1$,
 - και QPS συγκρίσιμο με IVFFlat,
- ενώ υπερτερεί ξεκάθαρα σε ποιότητα έναντι LSH, Hypercube και IVFPQ.

Συμπέρασμα για τη 2η εργασία

Από τα πειράματα στο SIFT φαίνεται ότι το Neural LSH:

- Μπορεί, με κατάλληλο build και επιλογή T, να φτάσει σχεδόν την ποιότητα του IVFFlat ($\text{Recall} \approx 0.99$, $\text{AF} \approx 1$) με παρόμοιο χρόνο αναζήτησης.
- Υπερτερεί σε ποιότητα (Recall & AF) σε σχέση με τις πιο “κλασικές” προσεγγιστικές μεθόδους της 1ης εργασίας (LSH, Hypercube, IVFPQ),
- ενώ διατηρεί τη δυνατότητα για πολύ υψηλό QPS όταν θυσιάσουμε λίγο Recall ($T = 1$).

Έτσι, το Neural LSH στο SIFT καταφέρνει να γεφυρώσει το κενό ανάμεσα στους “πολύ ακριβείς αλλά πιο αργούς” αλγόριθμους (IVFFlat) και στους “πολύ γρήγορους αλλά λιγότερο ακριβείς” (Hypercube, IVFPQ), το οποίο συνιστά τον στόχο της 2ης εργασίας.